

辞書をあらためて

必修事項のみ

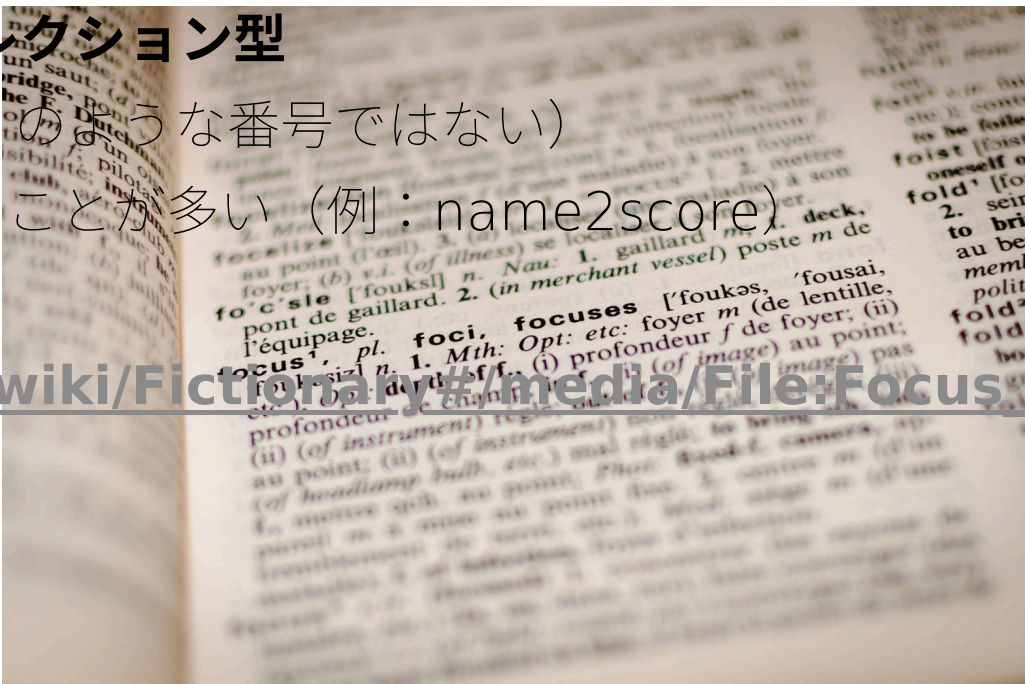
rkskmt

復習（dict とは）

- キーと値のペアで管理する **コレクション型**
- データは **キー** で管理する（list のような番号ではない）
- 変数名は **「x2y」** 形式にすることが多い（例：name2score）

CC0:

[https://en.wikipedia.org/wiki/File:Focus_definition_\(Unsplash\).jpg](https://en.wikipedia.org/wiki/File:Focus_definition_(Unsplash).jpg)



宣言

{ } の中に **キー： 値** をカンマ区切りで書くと辞書になる。

- キーは重複不可
- 値は何でもよい

Code

```
1 name2score = {"Yamada": 95, "Suzuki": 75, "Takahashi": 80}
2 print(name2score) # {'Yamada': 95, 'Suzuki': 75, 'Takahashi': 80}
```

💡 お作法

: の前は詰めて、**:** と **,** の後には **必ずスペースを空ける** のが綺麗に書くコツ。

アクセス（キー）

要素にアクセスするには **キー** を使う（list の **[0]** に相当）。

Code

```
1 name2score = {"Yamada": 95, "Suzuki": 75, "Takahashi": 80}
2 print(name2score["Yamada"]) # 95
3 print(name2score["Takahashi"]) # 80
```



要素の追加・上書き

dict[キー] = 値 で追加・上書きできる。

- 存在しないキー → **追加**
- 存在するキー → **上書き**

Code

```
1 name2score = {"Yamada": 95, "Suzuki": 75}
2
3 # 追加
4 name2score["Takahashi"] = 80
5 print(name2score) # {'Yamada': 95, 'Suzuki': 75, 'Takahashi': 80}
6
7 # 上書き
8 name2score["Suzuki"] = 70
9 print(name2score) # {'Yamada': 95, 'Suzuki': 70, 'Takahashi': 80}
```

⚠ **typo** に注意

追加と上書きが同じ書き方なので、**typo** でキーを間違えると新しいキーが勝手に追加される。

Code

```
1 name2score["Suzki"] = 60 # 上書きのつもりが新規追加！
```

5

走査（全要素へのアクセス）

dict には3種類の走査方法がある。**for** 文で書ける。

Code

```
1 # キーだけ
2 for key in name2score.keys():
3     print(key)
4
5 # 値だけ
6 for value in name2score.values():
7     print(value)
8
9 # キーと値を同時に（最頻出）
10 for key, value in name2score.items():
11     print(f"{key}さんは{value}点です")
```

Yamadaさんは95点です
Suzukiさんは70点です
Takahashiさんは80点です

💡 どれを使う？

- 両方必要 → **.items()**（実務ではこれが一番多い）
- キーだけ → **.keys()**（**for key in name2score:** でも同じ）
- 値だけ → **.values()**

6

その他 dict で知っておきたい基礎知識

- `in` 演算子で存在確認
- `.get()` で安全にアクセス
- 要素の削除と長さ (`del` / `pop` / `len`)
- ネストした dict
- dict と list の組み合わせ
- 辞書内包表記



in 演算子で存在確認

`in` 演算子で **キー** が存在するか確認できる。

Code

```
1 name2score = {"Yamada": 95, "Suzuki": 75}
2
3 if "Yamada" in name2score:
4     print("Yamada さんは登録済み")
5
6 if "Tanaka" not in name2score:
7     print("Tanaka さんは未登録")
```

💡 `in` はキーを見る（値ではない）

Code

```
1 if 95 in name2score:           # False
2 if 95 in name2score.values():  # True ← 値を見たいときはこう
```


.get() で安全にアクセス

dict[キー] は存在しないキーで落ちるが、**.get(キー)** は存在しなければ **None** を返すので落ちない。

Code

```
1 name2score = {"Yamada": 95, "Suzuki": 75}
2
3 print(name2score.get("Yamada"))    # 95
4 print(name2score.get("Tanaka"))    # None ← エラーにならない
```

第2引数で**デフォルト値**を指定できる。

Code

```
1 print(name2score.get("Tanaka"))    # None ← 第2引数なし
2 print(name2score.get("Tanaka", 0)) # 0    ← 第2引数あり（未登録なら 0 点扱い）
3 print(name2score.get("Yamada", 0)) # 95    ← 登録済みならその値
```

💡 in でガード vs .get()

- 「ない場合は処理を分けたい」 → **if key in dict:** でガード
- 「ない場合のデフォルト値が決まっている」 → **.get(key, default)** がスッキリ

9

要素の削除と長さ（del / pop / len）

del または **.pop()** で削除、**len()** でキーの数を確認できる。

Code

```
1 name2score = {"Yamada": 95, "Suzuki": 75, "Takahashi": 80}
2 print(len(name2score)) # 3
3
4 # del: 削除するだけ
5 del name2score["Suzuki"]
6 print(name2score)      # {'Yamada': 95, 'Takahashi': 80}
7 print(len(name2score)) # 2
8
9 # pop: 削除して値を取り出す
10 score = name2score.pop("Yamada")
11 print(score)          # 95
12 print(name2score)     # {'Takahashi': 80}
```

10

ネストした dict

値の中にさらに dict を入れると**入れ子（ネスト）**になる。 学生ごとに複数教科の点数など、複数の属性を持つデータを表現できる。

Code

```
1 students = {
2     "Yamada": {"国語": 80, "数学": 90, "英語": 70},
3     "Suzuki": {"国語": 60, "数学": 85, "英語": 95},
4     "Takahashi": {"国語": 75, "数学": 70, "英語": 80},
5 }
6
7 # Yamada さんの数学
8 print(students["Yamada"]["数学"]) # 90
9
10 # 全員の英語
11 for name, scores in students.items():
12     print(f"{name}さんの英語：{scores['英語']}点")
```

dict と list の組み合わせ

list の中に dict を入れる形は、表形式のデータを表すときの定番。（JSON や DB の1レコードのイメージ）

Code

```
1 students = [
2     {"name": "Yamada", "score": 95},
3     {"name": "Suzuki", "score": 75},
4     {"name": "Takahashi", "score": 80},
5 ]
6
7 for s in students:
8     print(f"{s['name']}さんは{s['score']}点")
```

💡

ネスト dict vs list of dict

- 名前から直接引きたい → **ネスト dict** （`students["Yamada"]`）
- 順序が意味を持つ・全員を順に処理 → **list of dict** （`students[0]`）

両方とも実務でよく使う。

辞書内包表記

リスト内包表記の dict 版。 **{キー: 値 for ... in ...}** の形。

Code

```
1 # 数 → その2乗 の dict を作る
2 squares = {x: x ** 2 for x in range(5)}
3 print(squares) # {0: 0, 1: 1, 2: 4, 3: 9, 4: 16}
```

条件付きも書ける。

Code

```
1 # 80点以上だけ取り出す
2 name2score = {"Yamada": 95, "Suzuki": 75, "Takahashi": 80}
3 passed = {k: v for k, v in name2score.items() if v >= 80}
4 print(passed) # {'Yamada': 95, 'Takahashi': 80}
```

キーと値を入れ替えるのも一行で書ける。

Code

```
1 score2name = {v: k for k, v in name2score.items()}
2 print(score2name) # {95: 'Yamada', 75: 'Suzuki', 80: 'Takahashi'}
```

13

まとめ

- dict は **キーと値のペア** で管理するコレクション（他言語ではハッシュマップ）
- アクセスは **dict[キー]**、ない場合は **KeyError** → **in** または **.get()** でガード
- 走査は **.items()** が最頻出、**.keys()** / **.values()** も使い分ける
- ネスト dict や list of dict で **表形式のデータ** を表現できる
- 辞書内包表記 **{k: v for ...}** で **for** + 追加を1行に圧縮できる

14

演習問題



Code

```
1 name2score = {"Yamada": 95, "Suzuki": 75, "Takahashi": 80, "Tanaka": 60}
```

上記の dict をコピーして、以下の操作を実装せよ。

- .get()** で **“Sato”** さんの点数を取り出す（存在しなければ **0** を返す）
- Tanaka** さんを **del** で削除
- .values()** と **sum** / **len** で、全員の **平均点** を表示

▶ 解答例を見る

15

演習問題（辞書内包表記）



以下の3つを **辞書内包表記** で実装せよ。

1. `range(1, 6)` から `{1: 1, 2: 4, 3: 9, 4: 16, 5: 25}` という「数 → 2乗」の dict を作る
2. `name2score` から **80点以上の学生だけ** を取り出した新しい dict を作る
3. `name2score` の **キーと値を入れ替えた** dict (`score2name`) を作る

① LUT（Look-Up Table）

問題1のように「**入力 → 計算結果**」を事前に dict に入れて引けるようにしたものを **LUT（Look-Up Table、参照テーブル）** と呼ぶ。毎回計算する代わりに `lut[x]` で引けるため、重い計算を **事前計算してキャッシュする** 用途でよく使われる。

古典的な例として、ゲーム機や組み込み機器では `sin` / `cos` の値を角度ごとに事前計算した LUT がよく使われる（毎フレームの三角関数計算は重いため）。

▶ 解答例を見る

16

演習問題（list of dict）



Code

```
1 # 要素：学生（0: A さん, 1: B さん, 2: C さん）
2 # キー：科目（"国語", "数学", "英語"）
3 students = [
4     {"name": "A", "国語": 80, "数学": 90, "英語": 70},
5     {"name": "B", "国語": 60, "数学": 85, "英語": 95},
6     {"name": "C", "国語": 75, "数学": 70, "英語": 80},
7 ]
```

上記の学生の国語・数学・英語の点数をもつ list of dict をコピペして、以下の操作を実装せよ。

1. **B さんの数学** の点数を表示
2. **C さんの3科目合計** を表示
3. **各学生の平均点** を `for` 文で順に表示

▶ 解答例を見る

17